CoCoA SONIC: The Gate Board
Executable Evidence, One Claim at a Time

V. Miranda
Code documentation manuscript
(Dated: Working draft)

This companion to the CoCoA SONIC code guide explains every gate registered in `ai/gates/board.py`, in exact board order. A gate is a named validation job whose required result is written before it starts, and the board is the ordered registry that runs them and reconciles their verdicts. The document covers the board's execution machinery, the anatomy of identity and smoke checks, worked examples, failure triage, the rules for adding a gate without creating a false green, and the safe-use warnings for running the board and preserving its evidence.

I. THE GATE BOARD, ONE EXECUTABLE CLAIM AT A TIME

This document explains all forty gates currently registered in `ai/gates/board.py`, in exact board order. A first-time contributor needs to know the gate name, the production path it executes, what its fixture means, which value determines the verdict and which plausible defect it can catch.

I.1. How to read each gate description

Every description below answers five questions: the claim made credible by a pass. The production path executed. The controlled fixture. The numeric, artifact or loud-error verdict and the deliberately wrong behavior that establishes catch power.

An identity gate usually uses a small analytic fixture and asks whether a transformation, save/rebuild cycle or composition preserves a known value. A smoke gate runs a short production-shaped calculation and asks whether independently implemented components connect. A golden run compares selected output from the current tree with a pinned earlier tree. Golden equality is useful for an advertised no-op mode. Behavior absent from the pinned tree lies outside that comparison.

I.2. Gate vocabulary in plain language

Fixture.: A deliberately small input prepared for a check. Its coordinates and expected answer are known before the production function runs.

Control.: A nearby case known to be valid. A positive control must pass. A negative control must be refused. Controls prevent a gate from becoming red merely because its runner is broken.

Mutation arm.: A deliberately wrong implementation or input form inserted only in the test. It represents a plausible defect, such as a transposed axis or an inflated tolerance.

Catch power.: Evidence that the gate turns red for the mutation arm. Agreement on the correct fixture without a discriminating wrong arm can be a coincidence.

Parity.: Equality between two paths that are required to implement the same function, for example eager and compiled evaluation. An independent answer supplies the scientific truth beside this path-agreement check.

Census.: A mechanical enumeration of every member of a source-code class, such as every model constructor or every assertion site. A census prevents a repair from covering only the first instance found.

Banner.: Human-readable run text reporting a resolved setting. A banner is useful provenance, but construction or numerical behavior must prove that the reported setting was actually used.

Sidecar.: A small companion file that gives meaning to a larger numerical dump, for example column names or an axis array. Row-for-row and artifact-identity rules determine which sidecar belongs to which payload.

Golden run.: Output pinned from an earlier executable tree for a behavior promised to remain unchanged. It is a regression reference. New mathematics requires an independent derivation.

Collapse bar.: A threshold that a model which learned no useful dependence cannot beat. It turns a short smoke from "the program ran" into a minimal liveness claim.

Provenance.: Facts identifying how a result was produced: source digest, resolved configuration, coordinates, dtype, device and artifact identity. Provenance permits a reviewer to reconstruct the claim's scope.

I.3. The board runner as a small execution system

`ai/gates/run_board.py` is the command-line runner. `ai/gates/board.py` is the declarative registry. Keep-

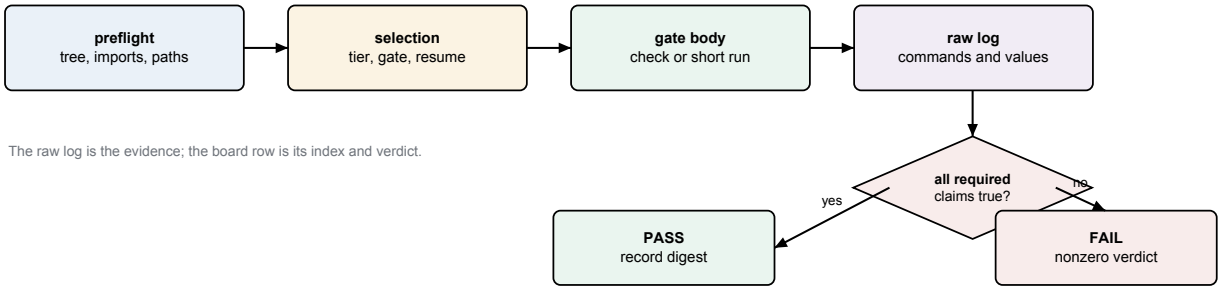

FIG. 1. The board's evidence flow. Preflight and selection determine what is eligible to run. The gate body produces evidence. The board converts all required claims into one explicit verdict.

ing them separate means the list of scientific claims does not reimplement subprocess, path or resume handling. A board entry supplies an identifier, title, tier, home documentation, capability requirements, dependencies and a Python function that expresses the gate body through a constrained context object.

The context object is the gate's interface to the outside world. It can resolve a configured path, launch a check script, launch a driver, record a message and assert an expected condition. Every gate process launch goes through the context. This restriction gives the runner one place to capture command lines, standard output, standard error, return status and elapsed time.

Standard output is the normal text stream a process prints. Standard error is a separate stream intended for diagnostics. Exit status zero conventionally means success and nonzero means failure. The runner records all three. Checking only for a phrase is insufficient if the process later exits nonzero. Checking only exit zero is insufficient if the expected scientific assertion never ran.

I.4. Reconciling declared and executed claims

The registry is beginning to represent an acceptance leg with two strings: a plain identifier and an anchor in its home note. The identifier names the specific observation, such as `stage-ram.exact-fit-boundary`. The anchor points to the paragraph that defines what result is acceptable. Registry validation can prove that the anchor exists and that no two legs share an identifier.

Those checks establish declaration only. Execution requires a terminal record for each declared leg. Suppose a gate declares three legs but a conditional return skips the third. The remaining two can pass and the subprocess can exit zero unless the runner compares the declared and executed sets. The complete mechanism therefore needs one terminal record per declared leg:

$$D = E,$$

where D is the set of declared identifiers and E is the set of identifiers with an executed terminal result: `PASS`, `FAIL` or `UNAVAILABLE`. `UNAVAILABLE` is non-green. It states that a required hardware or software lane was unexecuted. The denominator retains that lane. A complete pass still requires its result. A crash before a check script emits its record is likewise a missing leg and makes the gate red.

A three-leg example makes the set equality concrete. Suppose the registry declares

$$D = \{\texttt{roundtrip}, \texttt{wrong-axis}, \texttt{real-provider}\}.$$

The check executes the first two legs and then returns before contacting the provider. Its executed set is

$$E = \{\texttt{roundtrip}, \texttt{wrong-axis}\}.$$

Both emitted results can say `PASS`, but $D \neq E$: the missing `real-provider` leg makes the gate red. Every emitted identifier must also occur in D so it can be matched to a reviewed claim. `ctx.expect(label, condition)` is the in-process helper used by migrated gate bodies: `label` names the leg and `condition` is the boolean verdict. A false condition records failure and controls the gate result.

a. Structured-evidence coverage. For a gate that declares structured evidence, the runner checks that every human-readable `<gate-id>.<leg-name>` has one home-note anchor and exactly one terminal result. Unknown, duplicate and missing identifiers make that gate red. Eight gates still report one aggregate verdict without per-leg terminals: `finite-contract`, `finetune-identity`, `transfer-identity`, `save-rebuild-drift`, `cobaya-adapter`, `finetune-smoke`, `transfer-smoke` and `scalar-smoke`. For these gates, inspect the complete forced-rerun log and verify every advertised check before making a leg-specific claim.

I.5. Preflight protects expensive evidence

Preflight runs before GPU time. It checks the expected Git tip, watched-tree cleanliness, required imports and

configured data paths. A workstation log is evidence about one executable surface. If the worktree contains unrecorded modifications, the log cannot be reproduced from the named commit.

Import checks establish capability availability. Gate execution remains a separate requirement. Importing PyTorch proves that the module loads. CUDA kernels, compilation and mixed precision require their own executed checks. Opening a Cocoa data directory proves deployment availability. Adapter request correctness requires a separate executed check of the physical quantity.

a. Watched-tree behavior. The watched directory list now includes `emulator/`, `ai/gates/`, `compute_data_vectors/`, `cobaya_theory/`, `syren/` and the root drivers. The porcelain-status parser now preserves Git's leading two-column status field before examining individual lines. It excludes exactly `ai/gates/board_config.json`. An edit to neighboring `ai/gates/board.py` remains dirty. Self-test fixtures exercise the case in which the permitted config file is the first status line, because a whole- output `strip()` previously shifted that line alone. This clean-tree surface is shared by the report and the executed preflight decision.

b. One project root for every child. The runner resolves the project root from board configuration and injects it as `ROOTDIR` before launching a check or driver. It refuses an unresolved root and records the executed value in the raw log. The board self-test starts with one shell root and a different configured root, then verifies that the child opens files only below the configured root.

c. A warning leg also checks process success. The head-activation warning leg requires both a zero subprocess return status and the expected warning text. Its mutation fixture prints the correct warning and then raises, which makes the leg red. A printed phrase by itself is therefore insufficient evidence.

I.6. Selection, tiers, dependencies and resume

The runner can select gate identifiers, a tier or all gates at and after a named entry. Selection is validated against the registry before execution. A misspelled `-gate`, `-from` or `-force-rerun` identifier produces a nonzero usage error with nearby-name suggestions. The board self-test drives these public arguments and proves that an empty or dependency-skipped selection cannot report success.

A dependency states that one gate consumes an artifact or boundary produced by another. For example, adapter parity depends on successful save/rebuild. A dependency combines earlier list order with a confirmed verdict before the dependent body proceeds.

Resume skips a prior pass only when the executable and input digests, raw log and direct-dependency attempt lineage still match. It distinguishes stale code, stale inputs, stale dependencies, interruption, failure and dependency skip. Only current PASS is green. Force-rerun invalidates selected records without deleting unrelated logs.

a. Executable manifests on the live board. Every registered gate now declares an executable manifest. The runner derives membership from reviewed roots, persists per-file hashes and revalidates that membership on every invocation. Dynamic imports use reviewed waiver entries and subprocess targets outside the import graph appear as declared roots. A missing, truncated or digest-mismatched raw log becomes `stale-log`. A stored pass that predates its manifest becomes `pre-manifest`. Both states require a rerun.

I.7. Capabilities and where a claim is allowed to run

Each gate declares capabilities such as PyTorch, CosmoLike, Cobaya and GPU. The runner checks them before the body. Capability declaration prevents a missing dependency from being mistaken for a numerical failure. A missing capability produces the distinct non-green skip state described below.

Pure validation and NumPy algebra run on CPU. A claim about CUDA compilation must run on CUDA. A claim about Apple MPS float16 must run on Apple hardware. When a capability is optional, the board uses a distinct non-green skip state and states what remains unproven. Catching every exception and exiting zero would erase the difference between unsupported, broken and correct.

Hardware controls remain useful. A CPU or CUDA calculation that does not overflow records why that backend is only a control, while the affected MPS calculation supplies discrimination. The contract follows the executed dtype and device. Each accelerator needs its own evidence.

I.8. Anatomy of a pure numeric check

A pure check under `ai/gates/checks` is an executable Python file with a `main` function. It imports the production function, creates a bounded fixture, calculates an independent reference and accumulates named assertions. If any assertion fails, the process raises or exits nonzero.

The reference does not call the production helper under test. For a covariance contraction, the production side may use a banded implementation while the reference uses explicit loops and dense arithmetic on a tiny array. For a geometry, the reference may form the physical residual and direct inverse-covariance product. Agreement is valuable because the algorithms use distinct operations and failure modes.

Fixtures prioritize discrimination while retaining enough realism. A constant array can make correct and wrong axis orders coincide. An affine array can make several interpolation stencils exact. A strong fixture assigns distinct values to relevant coordinates and includes

Stored situation	What it means	What the runner must do now
Current `PASS`	Code, declared inputs and raw-log digest still match	Reuse the pass unless the operator explicitly forces a rerun.
Stale code	An executable member changed	Rerun. The old log remains historical evidence for its old executable surface.
Stale input	A declared fixture or configuration input changed	Rerun with the new input identity.
Pre-manifest	The gate now declares an executable manifest but its stored PASS predates it	Rerun. The old record's narrower digest cannot certify the declared surface.
Stale log	The raw log is absent, truncated, edited or fails its digest	Rerun. A database status cannot substitute for its evidence.
Stale dependency	The stored pass consumed an older attempt of a direct prerequisite	Rerun the child after the current prerequisite is green.
Interrupted	The body began but did not write a terminal gate verdict	Rerun the interrupted gate. Earlier independent current passes may remain reusable.
Stored `FAIL`	The last complete execution was red	Rerun after a bounded repair. Never resume it as green.
Dependency skip	A prerequisite was not green	Run or repair the prerequisite before the dependent gate.

TABLE I. Resume states and their operational meaning. "Resume" means reusing evidence from an earlier invocation and skipping the gate body in the current invocation. A digest is a compact fingerprint of bytes. Matching digests support identity, while a mismatch proves that at least one byte changed.

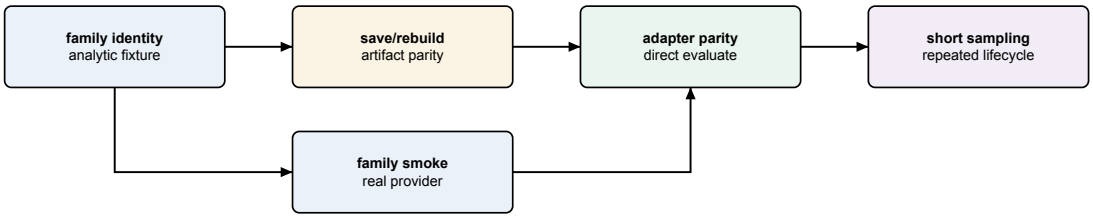

FIG. 2. A common evidence dependency. Identity establishes local mathematics, smoke establishes the real provider path, save/rebuild establishes persistence and adapter/sampling gates establish external use.

a control where the correct path is known to agree. The check prints actual and expected values so a red log carries both a diagnosis and a Boolean verdict.

I.9. Anatomy of a training smoke

A smoke YAML is small but production-shaped. It uses the real driver, resolver, loaders, model, loss, optimizer, evaluation and saving path. Small means fewer rows or epochs while retaining the production trainer.

A collapse bar requires the network to learn more than a dead baseline. For an analytic scalar target, the baseline may always return the training mean. The acceptance bar belongs below that baseline. Merely requiring a finite final loss allows an unchanged or disconnected network to pass.

Banners reveal resolved facts that are hard to infer from one metric: activation, normalization, phase, learning rate, scheduler cadence and source artifact. They are readbacks. A behavioral leg supplies the truth source and accompanies a banner whenever the configuration might be printed but ignored.

I.10. Test doubles and monkeypatch scope

A stub returns predetermined values. A fake implements a small working collaborator. A monkeypatch temporarily replaces the object referenced by production code. Replacing CAMB with a fake can prove that an adapter requests and combines keys correctly. It cannot prove CAMB returns the assumed convention.

Patch scope must match the lookup site. Replacing a class definition in one module does not affect another module that already imported and bound the old class. A gate patches the name looked up by the production function and restores it before later control arms.

A convention-honest fake makes confusable representations genuinely different. For CMB lensing it assigns different arrays to raw $C_L^{\phi\phi}$ and the scaled representation. If the fake used equal arrays, the wrong convention

would pass.

I.11. Mutation arms establish catch power

A mutation arm deliberately reinstates one plausible defect. The unchanged gate must fail under the mutation and pass under the production path. Examples include restoring width-squared chi-square tolerance, feeding scaled lensing spectra where raw spectra are required, using the old non-Gaussian weight and rebuilding under a drifted default.

The mutation is targeted. Changing many facts at once would not identify which assertion carries the catch power. Mutation evidence also detects a self-referential test: if actual and expected call the same helper, changing that helper changes both and the check remains green.

I.12. Tolerance is a derived part of the claim

Floating-point equality can be exact, absolute, relative or combined. Bitwise equality is appropriate when saving and loading the same tensor should perform no arithmetic. Relative tolerance is appropriate when the expected magnitude varies. Absolute tolerance supplies a floor near zero.

For a sum of w float32 terms, the chi-square domain band scales with accumulation width w and machine epsilon. A w^2 scale would admit materially negative scores. The band is computed from validated shape facts. The screened value has no authority to set it.

For example, float32 machine epsilon is approximately $\epsilon_{32} = 1.1921 \times 10^{-7}$. A kept residual width of $w = 780$ gives the declared domain band

$$\begin{aligned} b &= \max(10^{-6}, 32\epsilon_{32}w) \\ &\simeq 2.98 \times 10^{-3}. \end{aligned}$$

A computed score of -10^{-3} lies inside this roundoff allowance and is normalized to the exact physical boundary zero. A score of -10^{-2} lies outside it and is rejected before ranking. Replacing w by w^2 would widen the same example to about 2.32 and allow an impossible negative score to look perfect. The numbers show why "some small negative tolerance" omits required inputs: dtype and contraction width are part of the claim.

Every widened tolerance needs a measured valid control and a recorded derivation. Adjusting a tolerance until a red result turns green would let the defect define its own acceptance region.

I.13. Raw logs and evidence readback

Each raw log begins with gate identity, home documentation, Git commit and capabilities. It records commands and complete output. Named lines state the value behind each check and the final line is the gate verdict.

A compact illustrative excerpt is

```
gate: scalar-identity
commit: 1a2b3c4
leg scalar-identity.roundtrip: PASS
  max_abs=2.1e-07 tolerance=1.0e-06
leg scalar-identity.bad-name: PASS
  refusal=unknown scalar column
gate verdict: PASS (2 declared, 2 executed)
```

The first leg's label, measured value and tolerance explain a numerical pass. The second leg proves a malformed name was refused. "PASS" refers to the refusal behaving correctly. The final counts establish completeness only when the runner has reconciled the declared and executed identifiers. This excerpt teaches the intended record shape. Quoted workstation evidence must come from a raw log.

`logs/BOARD.md` is an index. A reviewer must use the raw evidence. A reviewer opens the raw log, confirms every advertised leg executed and reads values near thresholds. A pass at 4.99 percent under a 5 percent bar carries different operational risk from a pass at 0.1 percent.

Readback comes from the real consumer surface. The gate reopens the artifact or reads the adapter's public result, so the evidence covers the saved HDF5 record as well as the in-memory configuration.

I.14. Backlog tier: training and data contracts

1. *`ema-off-identity`: the absent feature is a true no-op*

Claim and path. Disabling the exponential moving average, EMA, leaves pre-EMA training byte-identical. The normal driver runs on the current tree and in a temporary worktree at the pinned pre-EMA commit. A worktree is a second checkout sharing Git objects. It does not switch the user's active checkout.

Fixture and verdict. Both runs use the short gate YAML. The scanner extracts declared epoch lines and compares their bytes. The current-tree run also exercises a functional smoke so an empty comparison cannot pass.

Catch power. Creating an averaged copy in the off branch, changing selection, consuming randomness or changing optimizer order alters a line. The next gate owns the EMA-on behavior claim.

2. *`ema-smoke`: the averaged state participates in a run*

Claim and path. With EMA enabled, the epoch-based horizon is used and a plateau rewind restores coherent state. A production training run uses `ema.horizon_epochs=3`. The normal optimizer, scheduler, selection and rewind code execute.

Fixture and verdict. The YAML causes a learning-rate cut during the short run. Banners must identify the horizon and rewind and the process must finish with the expected metrics.

Catch power. Removing the EMA update, converting the horizon incorrectly or rewinding live weights without their averaged partner removes a required observation.

3. `production-diagnostic`: *one report crosses several boundaries*

Claim and path. The production diagnostic command can stage cut data, train and create the expected report while exposing accurate size and coverage facts. The cosmic-shear driver runs with `-diagnostic` and uses production plotting code.

Fixture and verdict. The gate checks the model-class census, rows surviving the density window, printed train/validation sizes and shaded coverage triangle. A file suffix alone is not acceptance.

Catch power. Dead model branches, cuts applied after selection, misreported sizes or a plot omitting the declared window breaks a leg.

4. `single-phase-demotion`: *phase keys are handled deliberately*

Claim and path. A single-phase `resmlp` can receive the documented phase-shaped configuration without a traceback, while a structured model retains two-phase behavior. Both pass through the real resolver and driver.

Fixture and verdict. The single-phase fixture contains head-shaped keys and must train under the declared demotion. A two-phase control must still execute separate phases.

Catch power. Forwarding a head-only option into `resmlp` reproduces the old failure. Globally deleting phase settings makes the two-phase control fail.

5. `head-scheduler-override`: *the head owns its cadence*

Claim and path. A head scheduler replaces the relevant top-level settings and cuts the head learning rate on its own patience. The real two-phase resolver constructs the scheduler and the epoch loop feeds its validation metric.

Fixture and verdict. The override uses patience ten. The resolved banner must name it and the cut must occur on the matching cadence.

Catch power. A shallow merge that retains top-level patience, a scheduler attached to the wrong optimizer or a metric-free step shifts or removes the cut.

6. `eval-batch-invariance`: *scores are invariant under partitioning*

Claim and path. Validation scores remain invariant when the same ordered rows are partitioned differently. The check calls real `eval_val` with several batch sizes, including its production final-batch padding path.

Fixture and verdict. Per-row scores and aggregate metrics agree at relative tolerance 10^{-6}. Timing supplies a liveness signal outside the numerical verdict.

Catch power. Counting padded duplicates, leaking state between batches, batch-relative normalization or dropping a final short batch changes at least one per-row score.

7. `finite-contract`: *impossible scores never rank*

Claim and path. NaN, infinity and materially negative chi-square values are rejected at training, validation, diagnostics and warm-start parity boundaries. The PyTorch check calls the production reduction, `eval_val`, source diagnostic, safe square roots and parity helpers.

Fixture and verdict. Red fixtures include NaN in either parity arm, a nonfinite transfer surface, exact-fit zero, finite losses near the float32 maximum and finite negative chi-square. Valid controls include analytic positive square roots and tiny roundoff from an ill-conditioned positive-definite contraction. The allowed roundoff band scales with kept per-row width. Width-squared scaling is the rejected mutation.

Catch power. A mutation restoring the old width-squared band crowns a materially negative production-width score as perfect. Finite-only validation lets NaN comparisons evade threshold counts. On a supported compile lane, an exception produces failure. A green skip is forbidden.

8. `padded-head-identity`: *storage cells cannot become data*

Claim and path. Structured CNN and Transformer heads preserve the original physical coordinate of every kept output and keep artificial rectangle cells inert through every operation that could make them nonzero. The saved geometry and model checkpoint must describe the same map and mask. The writer verifies that agreement before staging either output file, and the reader verifies it independently before loading learned tensors.

Fixture and verdict. Small CPU fixtures use equal-count bins with different angular masks, a fully masked middle bin, nonzero-preserving activations, FiLM shifts, live convolutions and live attention. A second leg saves a model with a nonzero CNN correction, reopens it through the public reader and requires exact predictions and buffers. Authenticated checkpoints with a missing or disagreeing fixed layout buffer must be refused. A live

model paired with a different geometry must also be refused before staging, without changing a preceding valid artifact pair.

Catch power. Reconstructing positions from counts, masking only the initial scatter, applying only a token-level attention mask or trusting the checkpoint over the HDF5 geometry makes a focused witness fail while ordinary array shapes remain valid.

9. `board-selftest`: the runner reports what actually ran

Claim and path. The board's own selection, dependency, resume, atomic-status and evidence-map machinery cannot manufacture a green command from an empty, skipped, stale or interrupted run. The pure-Python check drives the real `run_board.main`, `select_gates` and resume helpers with small fake Gate objects.

Fixture and verdict. Fake gates pass, fail, skip through a failed dependency or raise an uncaught interruption. Unknown selectors and force targets must return nonzero with a suggestion. Mutating a gate body, referenced YAML, input config, evidence anchor or cited log must invalidate the corresponding record. For the currently migrated structured-evidence entries, every declared anchor must resolve to a real note location and every identifier must be globally unique. The live `-list` command reports the current registry size; a number copied into this guide would become stale. Most gates still lack individual leg declarations in the structured evidence map.

Catch power. A status-only resume rule, warning-only selector, log overwrite or board row whose note anchor does not exist makes one of the fake scenarios falsely green and is rejected. This gate tests runner control flow. Family gates supply the scientific evidence.

10. `generator-seed`: sampling owns one recorded random stream

Claim and path. Generator sampling is reproducible from one required integer seed. The seed constructs an owned NumPy `Generator` that is threaded through uniform draws, walker initialization, sampler moves and thinning. Process-global `np.random` state has no role in those draws.

Fixture and verdict. Two runs with the same seed produce identical draws and the chain header records that seed. Invalid seed types fail before sampling. A monkeypatch of global random functions must not affect the owned stream.

Catch power. One leftover global draw changes the same-seed result or triggers the monkeypatch. The pure gate proves ownership and local reproducibility. Append/restart replay and MPI worker-count invariance require their generator smoke legs.

11. `cli-strict`: misspelled flags stop before expensive work

Claim and path. Every public executable uses strict argument parsing. The check performs a source census and drives representative driver `main` functions through their real parsers while replacing the expensive training boundary with an instrumented stub.

Fixture and verdict. A valid command reaches the stub exactly once. The misspelling `-activaton` exits nonzero, names the unrecognized argument and reaches the stub zero times. All public entry points must use `parse_args`. The permissive `parse_known_args` form is forbidden.

Catch power. Silently retaining unknown arguments or manually discarding parser leftovers lets the misspelled option train a default model. The zero-call assertion exposes that behavior.

12. `family-first`: each driver owns one data family

Claim and path. A direct driver selects one physical data-block family before experiment construction. The cosmic-shear driver cannot become a CMB, scalar, background or matter-power run merely because the YAML contains that block, while each thin family wrapper accepts its own block.

Fixture and verdict. Wrong-family configurations fail at startup with both expected and received families. A clean cosmic-shear configuration reaches training and the four family wrappers reach their matching boundary. A census verifies that every cosmic-shear campaign pins the same family.

Catch power. Restoring automatic family dispatch makes a wrong-driver fixture run. Pinning only the single-run driver leaves a sweep or search census failure. The gate therefore covers the single-run, sweep and search drivers.

13. `stage-ram`: the memory decision counts every copy

Claim and path. `stage_source` budgets the compact parameter copy, compact target copy and row-reindex storage before choosing resident or file-backed staging. Parameter and target widths and dtypes are counted separately.

Fixture and verdict. A narrow target with a wide parameter table is chosen so target bytes alone fit but the coordinated copies exceed the budget. The real stage function must stay disk-backed. Resident and disk-backed controls return byte-identical selected rows, including an unequal-dtype case.

Catch power. A mutation restoring the old target-only estimate selects resident storage and fails the regime verdict even though its returned numbers remain correct. This separates resource truth from numerical truth.

14. `artifact-readback`: stored values keep their types

Claim and path. Artifact Boolean attributes are parsed by an explicit typed reader. Python truthiness is forbidden. The check exercises the shared reader and scans every artifact Boolean read site.

Fixture and verdict. A native Boolean is accepted, an absent key uses its named default. String or integer lookalikes, including the truthy string `"False"`, raise with file and schema context. The census requires all read sites to use the shared boundary.

Catch power. Replacing the reader with `bool(value)` turns `"False"` into true and can select drifted transfer weights. This pure gate proves type parsing and source coverage. The live save/forge/rebuild artifact leg remains a separate workstation obligation.

15. `diagnostics-domain`: invalid scores are refused before reports

Claim and path. Diagnostic producers apply the same finite, nonnegative chi-square boundary as training and evaluation before converting scores into coverage fractions or residual summaries. The check calls the real local-linear floor and CMB residual diagnostic and censes grid-family producer sites.

Fixture and verdict. Valid scores pass byte-identically. Negative roundoff inside the width-derived band becomes exact zero. Materially negative, NaN and infinite values raise with boundary, rows and band. A reachable one-coordinate negative floor is refused before `f_floor` is formed.

Catch power. Bypassing the screen recreates the former false `f_floor=0` "perfect" result. A valid diagnostic control prevents the repair from converting all small or difficult results into errors.

16. `berhu-loss`: the objective matches its piecewise law

Claim and path. The reverse-Huber objective has its declared value and derivative and a head can select it independently of a square-root trunk. A pure check calls the production reduction and autograd. A short two-phase run exercises schema resolution.

Fixture and verdict. Analytic points lie below, on and above the knot and cap. Required continuity is checked, while banners name square root for the trunk and capped BerHu for the head.

Catch power. Residual-norm units, a swapped branch inequality or a head resolved to the trunk loss change an analytic value or banner.

17. `loss-schema-equivalence`: syntax migration preserves meaning

Claim and path. Equivalent flat and nested loss settings resolve to the same run. The ordinary driver resolves and trains both forms.

Fixture and verdict. Selected numerical epoch lines reproduce the pre-migration result.

Catch power. Reading a key from the wrong nesting level, falling back to a new default or accepting conflicting spellings changes those lines. This equivalence check covers configuration migration. The analytic loss gate separately covers the numerical formula.

18. `berhu-anneal`: endpoints have mathematical meaning

Claim and path. BerHu annealing begins as square root, stays continuous at the hold boundary and reaches full BerHu at the declared endpoint. The shared production schedule supplies the blend coefficient.

Fixture and verdict. Values immediately around the hold boundary are compared, the epoch-15 fixture must have coefficient one and a no-anneal control preserves absent-key behavior.

Catch power. Reversing an increasing schedule, an off-by-one boundary or treating `const` as a generated ramp changes a sampled coefficient.

19. `ema-anneal`: the average becomes live at the intended time

Claim and path. EMA annealing excludes the deliberately poor early trajectory and evaluates averaged weights at the live point. The normal EMA update, shared schedule and model-selection path execute.

Fixture and verdict. A no-anneal control is compared with an annealed run. Output identifies the hold and metrics from the live averaged state.

Catch power. Updating during the hold, interpreting the horizon in the wrong unit or evaluating raw weights under an EMA banner breaks the transition.

20. `param-window-cuts`: selection follows physical cuts

Claim and path. A tight density window uses named parameter columns, shrinks the candidate pool by the independent count and then trains. The production cut resolver and staging selector operate on the real table.

Fixture and verdict. The expected surviving count is calculated independently and compared with the banner before the requested absolute number of rows is chosen.

Catch power. Cutting the wrong column, treating a missing side as zero, selecting before cutting or silently accepting too few rows changes the count or run verdict.

21. `triangle-shading`: visualization states the same domain

Claim and path. The optional coverage triangle shades all declared physical windows on their corresponding panels. The CPU check calls the production plotting helper and inspects its Matplotlib artists.

Fixture and verdict. Four nondegenerate synthetic windows have known panels and extents. The test counts fill objects and verifies coordinates.

Catch power. Shading only the last window, swapping axes or using untransformed bounds creates the wrong artist list.

I.15. New-features tier: model and family identities

1. `joint-training`: a live trunk and head really train together

Claim and path. With `freeze_trunk=false`, the second phase keeps both trunk and correction head trainable. The production phase transition sets `requires_grad`, creates optimizer groups and runs the joint graph.

Fixture and verdict. The banner states joint training and the check compares continuity across the phase boundary. Phase-two epoch time must sit above a frozen-trunk control because backward now traverses the trunk as well as the head. Timing supplies supporting liveness evidence. Parameter updates and gradient ownership carry the primary verdict.

Catch power. Leaving trunk parameters frozen, omitting them from the optimizer or evaluating a detached trunk can still let the head improve. Trainable-parameter counts and the control comparison expose those partial implementations.

2. `head-activation-pin`: one head may own a different curve

Claim and path. A structured head can pin a `multigate` activation while the trunk retains its model-wide family. Illegal phase combinations and head activations whose implemented derivative vanishes at zero are refused before training.

Fixture and verdict. The resolved model-spec banner names the head family and gate count. The parameter census must increase by the analytically expected activation tensors. A configuration that pins a head activation while requesting incompatible joint behavior must raise the teaching error.

Catch power. A resolver that prints the pin but still supplies the trunk factory to the head fails the parameter-count leg. Silently ignoring the illegal combination fails the error leg. This gate establishes construction. Separate behavioral head checks establish activation liveness.

3. `relu-tanh-norm`: fixed baselines use the requested normalization

Claim and path. Parameter-free ReLU and tanh activations can be constructed through the same factory interface and tanh runs with either shared affine or per-feature normalization as declared.

Fixture and verdict. Short tanh runs require the resolved norm in the banner and a descending loss. An absent-key control reproduces the default normalization behavior.

Catch power. Passing the feature width into `nn.Tanh`, sharing one mutable normalization instance across layers or silently selecting the default when `per_feature` was requested changes construction, parameter count or banner. Loss descent prevents a build-only false green, though separate head gates are needed for zero-initialization compatibility.

4. `weight-decay-census`: decay follows module role

Claim and path. AdamW decay applies exactly to weight matrices owned by `Linear`, `Conv1d` and `BinLinear`. It excludes biases, normalization gains and activation-shape parameters regardless of tensor shape.

Fixture and verdict. A model containing gated-power activations and structured layers is walked through the production optimizer factory. The check compares parameter object identities in the decay group with an independent owner-role census. A zero-decay golden control protects the off-mode behavior.

Catch power. The tempting shortcut `parameter.ndim > 1` incorrectly classifies some learned tensors and can miss custom weight owners. Because the expected set is named by owning modules, a shape-based mutation fails even if the total number of elements happens to match.

5. `npce-training`: the analytic base and neural correction compose

Claim and path. Neural polynomial-chaos emulation, NPCE, trains in both residual and ratio forms, persists enough state to rebuild, rejects mutually exclusive configurations and refits its base independently at each training-size sweep point.

Fixture and verdict. Short runs exercise both combination laws. Error fixtures request incompatible PCE/transfer or composition choices. A two-point N_{train} sweep verifies that each point receives a base fit from its own selected rows. Reuse of the first point's coefficients fails the second fixture.

Catch power. A module-global PCE, a cached term selection shared across points or a rebuild that restores only the neural residual changes the second point or

round-trip. Successful training alone would not prove the base was refit.

6. `finetune-identity`: warm start begins as the source function

Claim and path. Fine-tuning validates the source artifact, extends input geometry only according to the declared rule, pins the output geometry and produces epoch-zero predictions equal to the source before any optimizer step.

Fixture and verdict. The pure PyTorch check covers equal parameter sets, allowed extensions, anchor masks, constant or degenerate columns and configuration refusals. It compares both eager source predictions and the newly constructed training-side path at epoch zero.

Catch power. Recomputing the output center from new targets, mapping an extended input into the wrong column, loading weights non-strictly or applying an anchor before parity changes the epoch-zero result. Loud-error legs ensure an unsupported mismatch cannot be mislabeled as numerical drift.

7. `transfer-identity`: a zero correction is exactly the base

Claim and path. Frozen-base transfer slices the inputs required by the source, evaluates the source in its own encoded space, initializes a zero correction and composes gain or sum forms without altering the epoch-zero function.

Fixture and verdict. The check exercises several combinations of base-space and final-space composition, target packing, parameter surgery and family-kind refusals. A grid-family arm passes through its logarithmic law and treats outputs according to their family geometry.

Catch power. Applying the correction in the wrong coordinate space, double-encoding base inputs, unpacking the staged target at the wrong offset or testing a family mismatch only after another invalid fixture condition changes parity or the expected error.

8. `scalar-identity`: a named scalar survives every boundary

Claim and path. Scalar geometry, model state, save/rebuild, predictor output and automatic Cobaya provision preserve named derived quantities exactly.

Fixture and verdict. The check uses analytic scalar targets and exercises subsets, duplicate names, constant columns, wrong artifact kinds, trunk-only architecture constraints and fine-tune parity. Bitwise equality is required where no floating-point recomputation is warranted.

Catch power. Width-only assembly can return the right number of scalars in the wrong order. The named subset and duplicate-name legs catch that class, while constant-column and wrong-kind legs prevent a finite-looking but undefined geometry from passing.

9. `cmb-identity`: spectra, amplitude law and covariance agree

Claim and path. The CMB family preserves spectrum conventions, applies its amplitude/optical-depth law on the correct side of the metric, rebuilds exactly, evaluates roughness as specified, supports its structured head and constructs non-Gaussian band covariance weights correctly.

Fixture and verdict. The check covers ruled per-ℓ constants, forward/inverse amplitude laws, physical-χ^2 invariance, required parameters, roughness zero and high-frequency controls, fine-tune parity and a ResTRF identity-basis rebuild. Five covariance legs compare the production contraction with an independent known answer: exact contraction, deliberate old-weight miss, raw-versus-scaled lensing fixture integrity, width-three band projection and exact zero-band weight.

Catch power. Feeding scaled $[L(L+1)]^2 C_L^{\phi\phi}/(2\pi)$ where raw $C_L^{\phi\phi}$ is required creates a large known miss while the raw control matches. The round trip tests internal consistency, while the independent contraction exposes a consistently wrong convention.

10. `bsn-identity`: background functions obey their trained domain

Claim and path. Background-grid geometry, logarithmic-offset law, piecewise redshift windows, loud desert between windows, save/rebuild, adapter getters and fine-tune parity agree with closed-form flat-ΛCDM fixtures.

Fixture and verdict. Analytic $H(z)$ values span each trained window and approach its boundaries from both sides. Requests in the untrained gap must raise. Any interpolation across that gap fails the fixture. Derived distance relations are compared with independently integrated values.

Catch power. A generic interpolator can return smooth, positive and monotonic values in an untrained region. Those properties are insufficient. The explicit desert leg makes that plausible extrapolation red. The save/rebuild arm catches a law or window omitted from state.

11. `mps-identity`: a two-dimensional surface keeps its coordinates

Claim and path. Grid2d staging, geometry laws, constant-column pinning, matter-power base assembly, fine-tune parity, structured correction heads, stable streaming moments and staging-file lifecycle all preserve the declared (z, k) surface.

Fixture and verdict. Stub bases make the assembly algebra exactly known. Law-space rows are compared with independently transformed values. Stable-moment fixtures contain large offsets and float32 payloads and several chunkings must reproduce the stored-payload population variance. Lifecycle legs restage train and validation slots, verify superseded temporary files are unlinked and check cleanup after failure and sweep-point release.

Catch power. The retired sum/sum-of-squares variance changes with chunk order under cancellation. A precast float64 reference disagrees with the actual float32 payload. A gate that checked only `isinstance(memmap)` would miss leaked multi-gigabyte files. Absence checks after restage provide the discriminating evidence.

12. `geo-paths`: geometry classes have one import home

Claim and path. New artifacts persist class paths under the `emulator.geometries` package, legacy flat paths fail loudly and no live source import retains the retired geometry homes.

Fixture and verdict. The check creates fresh states, inspects their class markers, attempts legacy reconstruction and performs an untruncated tree census for old imports.

Catch power. Compatibility shims can make a local rebuild pass while allowing new files to continue publishing obsolete paths. Requiring both the new marker and the loud old-path refusal prevents that half-migration.

I.16. Save-and-sample tier: artifacts and external execution

1. `save-rebuild-drift`: an artifact is an executable recipe

Claim and path. Saving and rebuilding produces a bitwise-equal function for plain, factored, NPCE and convolutional-head models and the artifact remains governed by its persisted recipe when source-code defaults later drift.

Fixture and verdict. The check trains or constructs each supported form, saves the weight/HDF5 pair, rebuilds through `rebuild_emulator` and compares predictions before and after. For a structured head, persisted bin splits must reconstruct the same scatter layout. The check temporarily perturbs a code default and proves that the stored resolved value wins. Version-one and pre-persistence head artifacts must be refused with migration guidance.

Catch power. A weight-only test can pass while rebuilding a different geometry or layout. A current-default test can pass until the default changes. The deliberate drift arm establishes artifact facts as the owner of reconstruction. It perturbs ambient source defaults.

2. `cobaya-adapter`: inference repeats the training-side function

Claim and path. The Cobaya theory adapter reads named sampled parameters, calls the rebuilt predictor and returns the same physical prediction as the training-side model, including factored recombination.

Fixture and verdict. A parity check evaluates the same off-center cosmology through both paths and requires relative agreement at 10^{-6}. A Cobaya `evaluate` call then verifies the public lifecycle and a short MCMC smoke confirms repeated calls under the sampler.

Catch power. Reordered parameters, a missing decode, a factor law applied only in training or stale adapter state changes parity. The MCMC arm adds lifecycle evidence that one direct call cannot provide, while the direct known-input comparison supplies the precise numeric verdict.

3. `finetune-smoke`: a warm start can improve and republish

Claim and path. A real fine-tune begins at the source function, continues training on new data, writes source provenance and saves an artifact that rebuilds to the final fine-tuned function.

Fixture and verdict. The run consumes the board's own previously saved emulator. Before the first update, it prints and passes epoch-zero parity. It then completes, improves the declared metric, writes `finetuned_from` and survives save/rebuild.

Catch power. Loading source weights but recomputing incompatible geometry can still lead to eventual loss descent. The pre-update parity makes that path red immediately. Writing provenance without embedding the facts needed for reconstruction fails the final round-trip.

4. `transfer-smoke`: a frozen base and correction complete a run

Claim and path. A real transfer configuration loads the board's base artifact, composes a zero-start correction in the declared gain form, preserves epoch-zero parity, trains and saves a self-contained transfer artifact.

Fixture and verdict. The banner names the base path and composition form. The pre-train parity line

must pass, training must complete under its collapse bar and the saved HDF5 record must contain transfer provenance plus the embedded base facts needed for rebuild.

Catch power. Depending on the external base file after publication, packing the base and truth in the wrong order or composing in the wrong space may still produce a trainable correction. Parity and isolated rebuild separate those defects from ordinary optimization error.

5. `scalar-smoke`: *a complete analytic scalar example*

Claim and path. The scalar pipeline can learn a derived quantity whose formula is independently known, predict away from the training center, serve it through the scalar Cobaya adapter and generate its diagnostics.

Fixture and verdict. The target is analytic $\Omega_m h^2$. A short training must beat a collapse bar chosen below the mean-predictor baseline. An off-center cosmology avoids the degenerate case where every implementation agrees at the center. Cobaya `evaluate` readback is compared with the formula and expected diagnostic pages must be created.

Catch power. Returning the center, swapping parameter columns or advertising the scalar without inserting its calculated value can look plausible on a centered fixture. The off-center analytic readback makes each wrong path visible.

6. `cmb-smoke`: *real CAMB reaches training and inference*

Claim and path. The CMB generator, covariance script, training driver, artifact rebuild, Cobaya C_ℓ lifecycle and family diagnostics connect on real CAMB output. The non-Gaussian covariance path also executes with its normalization facts persisted.

Fixture and verdict. A smoke-scale CAMB run generates spectra. The covariance output is checked structurally, including non-diagonal blocks and provenance keys. Training must cross a relative collapse bar, the adapter must return the requested spectra and diagnostic pages must complete.

Catch power. This gate detects interface and convention failures that a synthetic fake cannot: CAMB key names, raw versus scaled spectra, actual coordinate ranges, covariance file shape and adapter lifecycle. The independent `cmb-identity` contraction supplies the numerical truth.

7. `bsn-smoke`: *background emulators agree with CAMB*

Claim and path. The background generator, two family trainings, saved predictors, Cobaya getters, derived-distance calculations and grid diagnostics work end to end against CAMB's own background solution.

Fixture and verdict. Real CAMB supplies $H(z)$ and reference distances at smoke-scale redshifts. A stale-cache tripwire ensures each generated row belongs to the current cosmology. The served values must agree within the declared two-percent smoke tolerance and the grid diagnostic pages must render.

Catch power. A generator can return a finite array cached from the previous point and a smooth interpolator can make it look reasonable. The generation token/tripwire and independent CAMB comparison jointly expose that class. Exact formula and desert-boundary behavior remain in the identity gate.

8. `mps-smoke`: *the matter-power surface completes EMUL2-shaped service*

Claim and path. The matter-power generator, two trainings, grid2d artifacts, Cobaya `get_Pk` lifecycle and residual-surface diagnostics connect on real CAMB data under the no-analytic-law path.

Fixture and verdict. CAMB produces a smoke-scale $P(k, z)$ surface on declared coordinates. Both trained pieces save and rebuild, the adapter returns surfaces on the requested grid and values are compared with the real reference within the smoke contract. The family pages must finish without materializing the production-size diagnostic cube.

Catch power. Transposing z and k, relabeling an equal-width axis, using the training axes for validation or assembling two artifacts in the wrong law space changes the surface. Stubbed identity legs prove exact assembly mathematics. This smoke proves the real CAMB and Cobaya interfaces.

I.17. Gate-by-gate failure triage

The first response to a red gate is to locate the smallest executed claim that failed. A tolerance change requires a numerical derivation. A rerun follows diagnosis. A gate with structured evidence exposes stable leg identifiers and the runner reconciles them with its declared set. For an aggregate-only gate, the check-script labels and raw log are the available evidence, so inspect the whole log before attributing a pass to one internal leg. The following guide identifies the first evidence to inspect for every board entry.

a. `ema-off-identity`. Diff the first unequal epoch line and determine whether the difference begins before model construction, at optimizer setup or at evaluation. A changed random draw suggests the off path consumed state. A changed metric with identical early banners suggests an off-mode branch entered the training or selection calculation.

b. `ema-smoke`. Read the resolved horizon and executed steps per epoch, then locate the learning-rate cut and rewind lines. If the cut exists but rewind is absent,

inspect scheduler-to-rewind control flow. If rewind exists but later metrics jump, compare which of live weights, optimizer moments and averaged weights were restored.

c. `production-diagnostic`. Separate a training failure from a report failure. Verify the cut-pool and size banners before opening the plotting traceback. If training succeeded but a page is absent, inspect the first diagnostic function that did not emit its completion marker. That missing marker identifies the first local report failure.

d. `single-phase-demotion`. Inspect the resolved architecture before the exception. A constructor receiving head keys indicates demotion happened too late. If the single-phase arm passes and the structured control fails, the resolver likely deleted phase information for every architecture.

e. `head-scheduler-override`. Compare the printed head patience with the epoch of the first cut. Correct printing with wrong cadence points to scheduler construction or metric stepping. Wrong printing points earlier, to precedence resolution. Confirm that the scheduler holds the head optimizer object. The trunk optimizer has a separate owner.

f. `eval-batch-invariance`. Find the first row whose score changes and whether it lies in the final batch. A difference only at the end implicates padding or trimming. Differences at every boundary suggest batch-relative state or reduction. Compare per-row arrays before comparing rounded medians.

g. `finite-contract`. Use the failed part label to distinguish producer, boundary, accumulation, tolerance, parity and compile arms. Print the offending row count and minimum. Do not weaken the predicate until the valid ill-conditioned control and the production-width negative mutation have been evaluated together.

h. `padded-head-identity`. Read the failed leg first. A layout failure points to scatter coordinates, per-operation masking or a fully masked row. An artifact failure points to geometry persistence or fixed model buffers. If saving fails, compare the live model with the resolved recipe before looking for a file error. If rebuilding fails, inspect the independent comparison before strict state loading. Do not replace the physical map with bin counts merely because the counts make the rectangle shape agree.

i. `board-selftest`. Classify the failure as selection, dependency, resume digest, interruption, log atomicity or evidence-map integrity. Inspect the fake gate's call count before the stored status: a body that did not run can never supply scientific evidence. For digest failures, change one input at a time and confirm which stale category the runner reports.

j. `generator-seed`. Compare the first differing draw and identify which sampling stage owns it. A difference before the sampler implicates uniform or walker initialization. A later difference implicates moves or thinning. Search for process-global random calls only after confirming the seed reached the owned generator.

k. `cli-strict`. Record parser exit status and expensive-boundary call count together. A nonzero traceback after the stub ran is too late. The typo must be rejected by argument parsing. A source-census failure names the executable that still accepts unknown flags.

l. `family-first`. Read the requested driver family beside the YAML's detected data block. If a wrong-family run reaches experiment construction, the driver omitted its pin. If a correct thin wrapper fails, inspect only its explicit family argument before changing shared dispatch.

m. `stage-ram`. Recalculate parameter bytes, target bytes, index bytes and budget separately. Agreement in values plus a regime mismatch locates the defect in accounting. The staging algebra has already matched. Compare resident and disk row coordinates before comparing payloads.

n. `artifact-readback`. Print the stored HDF5 type and value before conversion. A string that looks like a Boolean remains an invalid stored value. The reader must reject it without coercion. If the pure reader passes but a live rebuild fails, locate the read site that bypassed it.

o. `diagnostics-domain`. Identify the producer and its declared reduction width. Compare the raw score with the derived band before any plotting or availability logic. A within- band negative should normalize to zero. A material negative or nonfinite value should never reach a statistic.

p. `berhu-loss`. Identify the analytic region containing the first mismatch. A value mismatch at a knot suggests branch selection. A value match with gradient mismatch suggests a non-differentiable expression or misplaced detach. If only the training smoke fails, the analytic formula has passed and loss schema resolution is the next boundary to inspect.

q. `loss-schema-equivalence`. Diff resolved configurations before diffing epochs. The first differing leaf usually identifies an inherited default or wrong nesting level. If resolved records agree but epochs differ, look for a code path that still reads raw configuration after resolution.

r. `berhu-anneal`. Print the coefficient on the last hold epoch, first anneal epoch and final epoch. This distinguishes direction, boundary and endpoint errors. Then verify the loss uses that coefficient. A correct banner with unchanged analytic values means the schedule is disconnected.

s. `ema-anneal`. Inspect whether an averaged copy was updated during hold and which weights evaluation used at the live point. If schedule values are correct but metrics match raw weights, inspect the evaluation model selection. The schedule-value check has already cleared the horizon formula.

t. `param-window-cuts`. Resolve parameter names to column indices and independently recount each bound. A disagreement before combination indicates one formula or column. A disagreement only after combination indi-

cates mask composition. Confirm the absolute row count is applied to the surviving pool.

u. *triangle-shading.* List artist types, panel coordinates and bounds. A correct count with wrong panels is an axis-map failure. A missing fill with correct numeric masks is a plot ownership failure and does not invalidate the staging cut itself.

v. *joint-training.* Print trainable names and optimizer membership at phase entry. A parameter can have `requires_grad=True` yet remain absent from the optimizer. After one batch, compare trunk and head gradients and actual parameter deltas. Treat epoch loss as supporting evidence.

w. *head-activation-pin.* Compare the resolved head factory, instantiated class names and activation parameter count. If the banner is correct but count is not, the pin was recorded without construction. If the illegal configuration trains, the validator was bypassed or ran after allocation.

x. *relu-tanh-norm.* Inspect every residual block. The model's first norm cannot establish the remaining blocks. Shared object identity across layers means a factory returned one instance. For a finite but flat tanh run, inspect input range and normalization gains before assuming optimizer failure.

y. *weight-decay-census.* Report missing and unexpected parameter names with owning module types. A rank-based pattern appears as activation or normalization tensors in the unexpected set. A custom linear owner in the missing set means the role registry is incomplete.

z. *npce-training.* First identify residual versus ratio and the training-size point. If only the second point fails, compare PCE coefficient identities and selected term sets between points. If rebuild fails, inspect persisted domain, multi-indices, coefficients and combination law separately.

aa. *finetune-identity.* Compare source and new named parameter orders, then locate the first unequal stage: encoded input, first projection, network output or decode. An output difference with equal network coordinates implicates geometry. A difference at the first projection implicates extension or weight surgery.

bb. *transfer-identity.* Print base input slices, base-space output, correction and final composition in that order. This exposes double encoding and wrong-space composition. For an unexpected error message, verify the fixture is invalid only in the dimension under test.

cc. *scalar-identity.* Read named outputs beside their columns. Equal width is not enough. For a constant-column failure, inspect the stored dtype after scale construction. A positive float64 scale may underflow to zero in float32.

dd. *cmb-identity.* Identify spectrum, multipole and representation before examining tolerance. For covariance failures, compare raw fixture arrays, independent per-band accumulation and production weights. For amplitude failures, divide the reported metric by the expected physical factor to expose an accidental multiplicative law.

ee. *bsn-identity.* Classify the red point as trained window, exact boundary or desert. A finite desert return is a domain-policy failure even if monotonic. A trained-window numeric mismatch should be traced through output law, interpolation and distance conversion in that order.

ff. *mps-identity.* Separate coordinate, law, moment, assembly and temporary-file legs. For moment failure, record chunk order and compare against the stored float32 payload in its stored representation order. Precast data uses the wrong reference. For cleanup failure, list live train and validation slots and which restaging event should have unlinked each path.

gg. *geo-paths.* Inspect the class marker in a newly written state before chasing imports. If the marker is current but the census is red, a live source still depends on a retired home. If a legacy path loads, locate the compatibility shim that made the required loud refusal impossible.

hh. *save-rebuild-drift.* Compare predictions at four boundaries: pre-save eager, rebuilt eager, rebuilt under drifted defaults and family public predictor. Then compare state keys, shapes and resolved construction facts. Non-strict loading is never a repair for an unexplained missing key.

ii. *cobaya-adapter.* Print named sampled values in artifact order and compare the predictor result before adapter assembly. If direct parity passes but repeated sampling fails, inspect per-call state reset and provider verdict order. If parity itself fails, inspect units, axes and decode before MCMC.

jj. *finetune-smoke.* Treat failed epoch-zero parity as a construction failure and stop before interpreting later improvement. If parity passes but save/rebuild fails, compare fine-tune provenance and inherited architecture facts. If training does not improve, inspect learning rate and trainable census.

kk. *transfer-smoke.* Verify the base digest and family identity, then epoch-zero parity, then correction learning. A missing embedded base is a publication failure even when the run succeeds locally with the external source still present.

ll. *scalar-smoke.* Evaluate the analytic formula at the exact off-center input printed in the log. If direct prediction agrees but Cobaya readback does not, inspect automatic provides and state insertion. If both return the center, inspect input ordering and encoding.

mm. *cmb-smoke.* Follow generator, covariance, training, rebuild, adapter and diagnostics markers in order. The first absent marker locates the interface boundary. Use `cmb-identity` to judge a normalization disagreement. The smoke's generated covariance serves its connectivity fixture.

nn. *bsn-smoke.* Check the stale-cache tripwire before comparing CAMB values. A tripwire failure invali-

dates every later numeric agreement. For a getter mismatch, compare requested redshift sequence with artifact axes before inspecting interpolation.

oo. `mps-smoke`. Confirm the stored z and k sequences on both artifacts and the request before comparing flattened arrays. A transpose can preserve shape and summary statistics. If coordinates agree, inspect lawspace assembly and only then the real-provider tolerance.

I.18. Worked example: a finite-contract gate

Consider the claim that validation must refuse a finite negative $\Delta\chi^2$. First identify the public boundary. The relevant function is `eval_val`, because it creates per-row validation scores and ranks epochs. The gate calls `eval_val` directly and therefore proves that the public evaluator uses the predicate.

The fixture uses a small test double returning a score vector with a negative value well outside the derived roundoff band. All other inputs are finite and validly shaped. This isolates one invalid property: an exception cannot be misattributed to NaN input, shape mismatch or an absent model parameter.

The expected result is a typed exception naming the validation side, affected row count, minimum and allowed band. The gate requires its own typed classification before low-level square-root work. A warning afterward arrives too late.

Next comes a near-zero valid control. A small positive-definite covariance produces a tiny negative roundoff under finite precision. The shared predicate normalizes a value inside the derived band to exact zero. This valid control proves that the repaired gate still accepts legitimate roundoff.

The mutation restores the retired width-squared band. At realistic output width, the chosen negative score then falls inside the wrong band and becomes zero. The mutation must make the gate red. Finally, the same predicate is exercised through training reduction and source diagnostics. One helper owns the rule, while integration legs prove every public producer calls it:

$$\begin{aligned}\text{public boundary} &\to \text{red fixture} \to \text{typed verdict}\\ &\to \text{valid control} \to \text{mutation}\\ &\to \text{boundary census}.\end{aligned}$$

I.19. Worked example: identity and smoke for the CMB family

The CMB family needs both an end-to-end smoke and an identity check. The smoke can run CAMB, write spectra, construct a covariance, train and serve C_ℓ. If every stage uses the same wrong lensing convention, the path can be internally consistent and scientifically wrong.

The identity fixture makes raw and scaled lensing spectra different. Let the fake raw spectrum be nonconstant $C_L^{\phi\phi}$. Independently form

$$\widetilde{C}_L^{\phi\phi} = \frac{[L(L+1)]^2}{2\pi} C_L^{\phi\phi}.$$

The fake returns each only under its proper request. Production receives the raw array. A separate explicit loop computes the expected band contribution. Agreement establishes the correct arm.

For catch power, feed the scaled array into the raw slot without changing the reference. The band weight must miss by a large recorded margin. This is stronger than asserting that the arrays differ: it proves the final contraction is sensitive to the convention.

The real-CAMB smoke checks that the generator requests available keys, lengths and multipole sidecars agree, training consumes the covariance, the artifact answers Cobaya's requested sequence and the non-Gaussian configuration executes its declared path.

The logs meet through persisted provenance. The smoke asserts which path ran. The identity asserts that this path has the right mathematics. Each log uses an independent numeric truth source.

I.20. Worked example: artifact drift

An artifact combines tensors with instructions for interpreting them. A drift test changes an ambient code default after writing the artifact. Suppose a model was saved with activation H, three gates and affine normalization. The check records an off-center prediction y_{before}.

The source default is monkeypatched to another allowed setting. Public rebuild must read the persisted resolved configuration and produce y_{after}. Because this fixture requires no new arithmetic, the comparison can be bitwise:

$$y_{\text{after}} = y_{\text{before}}.$$

The off-center input matters. At zero input or identity initialization, several architectures coincide and a drifted build can pass accidentally. The check also inspects class, parameter names, output geometry and head layout. Prediction equality for one row can be coincidental. State identity plus several discriminating predictions localizes failure.

An old artifact missing a required construction fact is refused with a migration instruction. Guessing from current defaults makes the file load today and change meaning tomorrow. Loud refusal is correct when reconstruction cannot be proven.

I.21. How to add a gate while preventing a false green

Start with one sentence that can be false. For example: *the rebuilt predictor returns the same named spectrum on*

the same multipole sequence. Avoid a broad goal such as *test CMB*. A narrow claim determines the public boundary, fixture and verdict.

Then follow this construction order.

1. Name the production owner and call its public function. If a private helper also needs a unit check, retain the public integration leg.
2. Build the smallest fixture preserving the distinction. Use nonconstant, nonsymmetric values when order matters.
3. Derive the expected answer independently. Write down units, axis, dtype, device and shape before calculating it.
4. Add a valid edge control matched to the claim: exact zero, one row, final short batch or an ill-conditioned positive-definite matrix.
5. Add a mutation or deliberately wrong form and verify the unchanged assertion fails it.
6. Give every failure a typed or named verdict. A red leg requires the warning and a nonzero exit.
7. Declare hardware and external capabilities. Do not hide an unavailable required lane inside a broad exception handler.
8. Register the gate with home documentation and actual artifact dependencies.
9. Run the check directly and through the board. Direct success plus board failure usually indicates deployment, path, capture or capability metadata.
10. Temporarily break the intended production line and observe the board turn red before accepting the new gate.

I.22. Common false-green patterns

a. Expected and actual share one helper. The test repeats the defect. Replace the expected side with explicit algebra, a closed-form solution or a differently structured reference.

b. The fixture erases the distinction. Zeros, constants, identity matrices and centered cosmologies make wrong paths agree. Keep them as edge controls and include a discriminating fixture.

c. Any exception counts as success. An always-raising implementation passes. Require the documented exception type and message facts, then include a healthy in-range control.

d. A banner substitutes for behavior. The resolver can print a setting that construction ignores. Pair the banner with parameter counts, state readback, a numerical transition or a changed weight.

e. Only aggregates are compared. Permuted or transposed rows can preserve a mean. Compare per-row and per-coordinate arrays before reducing.

f. A skip exits zero. The board records a pass without evidence. Use a distinct skip status that cannot contribute to a full-board denominator.

g. Tolerance follows the failure. The defect defines its own acceptance. Derive tolerance from dtype, operation count and valid controls before looking at the red value.

h. The smoke is its own reference. Internal consistency replaces truth. Pair the smoke with an identity or an independent external implementation.

I.23. A compact evidence checklist

For each raw log, a reviewer checks:

1. Confirm that the header identifies the exact commit and watched surface.
2. Confirm that every promised leg prints a start and verdict.
3. Find the production public boundary in the execution record.
4. Record actual, expected, tolerance, dtype, device, shape and coordinates.
5. Confirm that the valid control is close for the stated reason.
6. Confirm that the mutation fails through the intended assertion.
7. Exclude skips from the pass count.
8. Use artifact or adapter readback to show that the consumer sees the same resolved fact.
9. Record a thin margin to threshold explicitly.
10. Reproduce the evidence without untracked workstation state.

I.24. Reading one registry entry field by field

A `Gate` entry is data describing execution. Its `id` is the stable command-line name and log-file stem. Changing an identifier breaks resume continuity and external run instructions, so titles may improve while identifiers remain stable.

The `title` is human-facing. It states the claim in ordinary language. The `tier` controls grouping and scheduling. Executed checks determine pass semantics. The `home` points to the maintained design note that owns the acceptance contract. The optional map text gives a reviewer a more precise route into that note, but executable assertions remain the verdict.

The `run` field holds a function object and registers work for later. The runner calls it with the context for the current log, configuration, capabilities and dry-run state. This delayed call is why constructing the board at import time need not launch tests.

`needs` is a tuple of capability names. A tuple is an immutable ordered Python sequence. The runner iterates it before the gate body. `deps` is a separate tuple of gate identifiers whose accepted products or boundaries are required. Capability availability and scientific dependency are separate relationships.

The optional golden-worktree commit identifies a comparison tree for a no-change claim. Its expected-answer scope is limited to unchanged behavior. If a feature is new, an older commit lacks the behavior and cannot establish its mathematics. The gate uses analytic, mutation or real-provider evidence for that case.

Inside the body, `ctx.run_check(path)` launches a repository check script and returns its exit status and captured output as ordinary values. The body passes the status into `ctx.expect`. `ctx.expect` records a named check and changes the gate verdict when its boolean condition is false. The label names the scientific claim represented by the Python comparison.

Dry-run mode follows the same registry and dependency traversal but prints commands without executing them. A gate body must not interpret absent dry-run output as failed science. Conversely, dry-run success is never stored as a gate pass because no assertion values were produced.

I.25. Choosing the right evidence form

a. Use exact identity when no arithmetic should occur. Examples include loading the same tensor bytes, an off mode that bypasses a branch and a zero correction whose composition is algebraically the base. Exact comparison gives the narrowest contract and catches accidental casts or reordered operations.

b. Use a tolerance when the correct path performs floating-point work. The tolerance is derived from dtype and operation structure. Record absolute and relative parts explicitly. Compare in the dtype in which the verdict is defined. Upcasting after a float32 computation cannot recover lost information.

c. Use a golden run for a promised no-op migration. Schema rearrangement and disabled optional features are good candidates. Pin the base commit and select stable, science-bearing output lines. Do not use a golden run to freeze a known defect or to replace a new feature's reference.

d. Use an analytic fixture for a mathematical law. Closed-form scalar outputs, polynomial surfaces, diagonal covariance and simple distance relations make good fixtures. Exercise off-center and nonconstant values so competing formulas do not coincide.

e. Use a fake for an external protocol. A fake can record requested keys, enforce call order and return generation-tagged arrays. It proves the code handles the protocol it was given. Follow with a real-provider smoke to show the protocol matches the installed collaborator.

f. Use a smoke for production connectivity and liveness. A smoke should cross the same public entry points used in production and beat a baseline that a dead component cannot beat. Reduce rows and epochs to keep it short. Retain the component under test.

g. Use mutation when a plausible wrong path also looks reasonable. Axis relabeling, stale cache, raw/scaled convention, tolerance inflation and configuration printing without construction can all yield finite outputs. A targeted mutation proves which assertion distinguishes them.

I.26. Why gate names pair identity and smoke

The family pairs are an intentional reading aid: `scalar-identity`/`scalar-smoke`, `cmb-identity`/`cmb-smoke`, `bsn-identity`/`bsn-smoke` and `mps-identity`/`mps-smoke`. The shared prefix says the gates refer to one physical family. The suffix says which evidence layer they own.

Warm-start and transfer use the same pattern. Their identity gates establish epoch-zero composition and error paths without external training cost. Their smokes consume a saved artifact and prove improvement, provenance and final rebuild. If the identity is red, running the smoke spends accelerator time on an undefined starting function. If identity is green but smoke is red, the failure lies in training, persistence, deployment or the real data path while the core algebra has already passed its isolated check.

Some cross-cutting features use an off-identity/on-smoke pair. EMA first proves that absence preserves old behavior, then proves the new averaged state is live. The name states the two distinct claims that the feature needs.

The board order places cheap, pure identities before dependent expensive smokes where practical. This is fail-fast scheduling: discover a local algebra or schema failure before allocating a workstation for an end-to-end run. Order never allows a smoke pass to override a failed identity. Both remain separate verdicts in the final board.

I.27. From a red log to a bounded repair

A repair begins at the first failed invariant. The final traceback supplies path evidence. Reproduce the smallest failing gate directly. Preserve the original raw log, then instrument values without changing the acceptance predicate. Once the mechanism is understood, state a bounded implementation contract and add the counterexample that current code gets wrong.

The implementer changes the production owner and the gate together only when the existing gate lacks the required distinction. A gate must not be relaxed to match the implementation. If a previously accepted behavior was itself wrong, the record states that the acceptance is reopened and replaces its reference with independently justified truth.

After the repair, run the direct gate, its mutation arm, dependent family identity and the relevant smoke. Then run the complete board when the changed surface can affect other families. Record the exact source commit beside the raw logs. A reviewer can then reconstruct this chain:

$$\begin{aligned}\text{counterexample} &\to \text{root cause}\\ &\to \text{bounded change}\\ &\to \text{discriminating leg}\\ &\to \text{regression evidence.}\end{aligned}$$

This discipline keeps a red result useful. The goal is a true scientific claim with executable evidence that catches the same failure class later.

I.28. Mapping gates to the pipeline boundaries

The board can also be read horizontally, by the boundary each gate protects. At the configuration boundary, `single-phase-demotion`, `head-scheduler-override`, `loss-schema-equivalence`, `head-activation-pin`, `relu-tanh-norm`, `cli-strict` and `family-first` prove that commands and text become the intended constructible objects for the intended physical family. Their common risk is a value that is accepted or printed but does not control construction.

At the generation and data-selection boundary, `generator-seed`, `stage-ram`, `param-window-cuts`, `triangle-shading` and `production-diagnostic` connect reproducible cosmology draws, physical support, staged rows, resource policy and the human coverage report. The plot gate checks the domain shown in the report. The selection gates check the enforced row set.

At the objective boundary, `berhu-loss`, `berhu-anneal`, `finite-contract` and `eval-batch-invariance` protect the meaning of a score. `diagnostics-domain` extends the same boundary to post-training estimators. They distinguish the per-row physical metric from its training transform and they ensure batching, invalid values or schedule position cannot redefine which epoch looks best.

At the optimizer-state boundary, `ema-off-identity`, `ema-smoke`, `ema-anneal`, `weight-decay-census`, `joint-training` and `head-scheduler-override` establish which parameters move, which auxiliary state advances and which metric controls the learning rate. This group covers state transitions and tensor formulas.

At the representation boundary, the scalar, CMB, background and matter-power identity gates protect names, axes, units, geometry laws and family-specific assembly. Their smoke partners then carry the verified representation into the real physics provider. `geo-paths` protects the Python class identity that reconstructs those representations.

At the reuse boundary, NPCE, fine-tune and transfer gates establish how an existing function participates in a new model. Identity gates prove the initial composition. Smokes prove that the composed model trains and publishes. Their central invariant is that reuse starts from a known function whose identity is established before its tensors are loaded.

At the publication boundary, `save-rebuild-drift` proves that the saved recipe owns reconstruction, `artifact-readback` protects typed metadata and `cobaya-adapter` proves that the external consumer sees the same function. Family smokes add repeated lifecycle calls. These gates close the final gap between an in-memory result and a theory product used by inference.

Finally, `board-selftest` protects the evidence-control boundary itself: selection, dependency, status, digest, log and note-map machinery. Scientific truth comes from the family gates. The self-test prevents the runner from reporting a claim as executed when it was empty, skipped, stale or interrupted.

This horizontal reading reveals coverage gaps. A new component that changes configuration, representation, training state and publication needs evidence at each affected boundary. One end-to-end smoke covers its executed path, while the earlier boundaries need their own claims. A construction identity and a public-driver reachability check also cover separate questions. The gate plan is complete when every changed arrow in the ownership chain has a named executable claim.

I.29. Running the board and preserving the evidence

The workstation operator begins in an environment where Cocoa, PyTorch and the relevant external packages are importable. The first command is

```
python ai/gates/run_board.py --check
```

This performs preflight without launching GPU work. A clean preflight states which repository commit, deployment configuration, watched paths, imports and data roots will govern the run. If any fact is wrong, fix the deployment and run preflight again. Do not treat an intended different tree as close enough.

Next,

```
python ai/gates/run_board.py --dry-run
```

prints the selected order, dependencies, capabilities and commands. Dry run is the place to catch a wrong YAML path or accidentally broad selection. Because it produces no scientific results, it never writes pass verdicts.

The full command

```
python ai/gates/run_board.py
```

streams each command into its per-gate raw log. The terminal output is useful for monitoring, but the log file is the durable record. If the process is interrupted, completed matching passes remain eligible for resume and the in-flight gate runs again.

For a bounded investigation, `-gate` selects named gates and `-from` selects the suffix beginning at one gate. `-tier` selects a registry tier. Force-rerun applies only to explicitly named matching gates unless the all-selected form is requested. Before trusting the plan, the operator reads the printed selected count and identifiers. Zero selected gates returns an error. It can never pass as a no-op.

After the run, begin with the summary to locate red or skipped entries, then open every corresponding raw log. For a full acceptance run, sample green logs as well, especially thin-margin and hardware-specific gates. Confirm the last gate verdict, but also confirm every advertised leg appears above it. A gate body that returned early could otherwise produce an incomplete log whose last executed check happened to be green.

Logs are committed only after the operator verifies that they name the intended commit and contain no unrelated workstation secrets. The log commit adds evidence metadata while leaving the tested source commit fixed. The evidence note names both commits. If source changes after the run, the old logs remain historical evidence and a new run is required for the new executable identity.

When a gate needs CUDA or another workstation-only capability, the repository maintainer supplies the check under `ai/gates/checks`, registers it in the board and gives the operator an exact selection command. The operator is not asked to paste an untracked Python snippet into a GPU shell. Keeping the check in the repository makes its source part of the watched evidence surface and lets later board runs repeat it.

The final handoff reports counts only after excluding skips and unexecuted gates. On the board described by this document, a phrase such as 40/40 means forty registered required gates executed and passed on the recorded surface. If the environment lacks a required lane, the honest result is fewer certified claims plus an explicit outstanding capability. A smaller denominator would falsely present a complete board.

I.30. Scope of a full green board

A green board means that every registered executable claim passed on the recorded source tree, configuration, dependencies and hardware. Unregistered states remain outside that evidence. The board therefore persists the Git commit and should bind resume records to a digest of the complete executable surface: production modules, gate bodies, fixtures and relevant configuration.

The strongest pattern is paired evidence:

1. an identity fixture proves an exact mathematical or serialization statement.

2. a mutation or deliberately wrong form proves the assertion can fail.

3. a smoke run proves the real external components reach the verified boundary.

4. artifact readback proves the published object carries the facts the executed path used.

When only one layer is available, the log should say exactly what remains unproven. A CPU identity cannot certify CUDA compilation. A CUDA smoke cannot establish an independent analytic truth if its reference repeats the production helper. A PDF existing cannot prove its plotted coordinates. The board is useful because those claims stay separate and named.